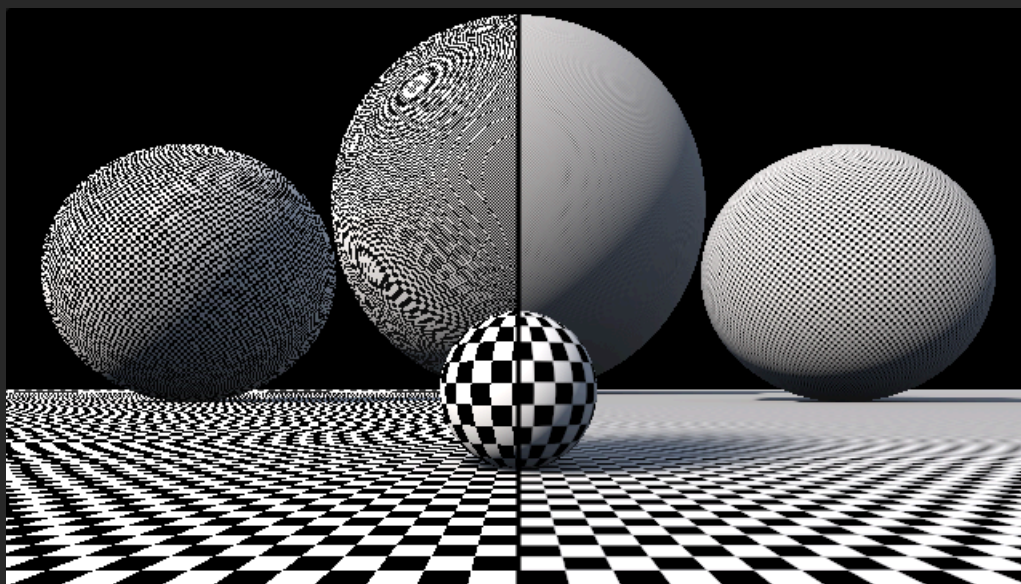


Inigo Quilez :: articles :: filtering the checkerboard pattern - 2017



Intro

The checkerboard pattern is often seen in computer graphics tools and papers as a placeholder for more sophisticated patterns or textures. Still, when it is employed, there's no reason to be sloppy about it - users still expect the images to look high quality, naturally including properly antialiased. One way to do so might be so simply store the actual checkerboard pattern in a mipmapped texture, in which case filtering will be performed the usual way. However, for different reasons you might want to avoid using textures for the pattern and do it procedurally. Then this article might interest you, since it explains how to perform filtering of checkerboard patterns analytically (without precomputed textures or mips). This article is heavily inspired by the book "Advanced Renderman" by Anthony A. Apodaca and Larry Gritz (1999) which tackles the filtering of pulse signals (page 273). You can also find a summary of it in the [Renderman Shading Language documentation](#).



Left: naive checkerboard. Right: analytically filtered checkerboard.

The Starting Point

We start by having a simple checkerboard patter what will alias under normal circumstances:

```
// checkers, in mod form
float checkers( in vec2 p )
{
    vec2 q = floor(p);
    return mod(q.x+q.y,2.);
}

// checkers, in xor form
float checkers( in vec3 p )
{
    ivec2 ip = ivec2(round(p+.5));
    return float((ip.x^ip.y)&1);
}

// checkers, in smooth xor form
float checkers( in vec3 p )
{
    vec2 s = sign(fract(p*.5)-.5);
    return .5 - .5*s.x*s.y;
}
```

These three variations produce the same pattern. But for our purposes, we'll chose the third form, since it is easily extended into a smooth function. To do so, not that the **s** is a square signal oscillating between -1 and 1 in both the **x** and **y** coordinates. The multiplication on the second line of code produces the xor pattern since neg*neg=pos, neg*pos=neg, pos*neg=neg, pos*pos=pos. The 0.5 scale and bias brings the value for the pattern from [-1,1] back to [0..1].

Now, the point of picking this expression for the checkers patter is that we can hopefully analytically filter the square signal **s** and get some filtered checkers pattern.

Filtering

Filtering for us basically means that where we normally would evaluate/sample the checkers pattern with some particular **uv** texture coordinates in our pixel shading function, instead we are going to consider all the checkers pattern area covered by the pixels under shading. When considering the whole area, we will do some weighted average of all the content of the area. The size of the area covered by the pixel together with the weights assigned to each point in the area determines the so called **kernel**. If all weights are the same, we are doing a box filter. Since that's the simplest of all, lets start with that one.

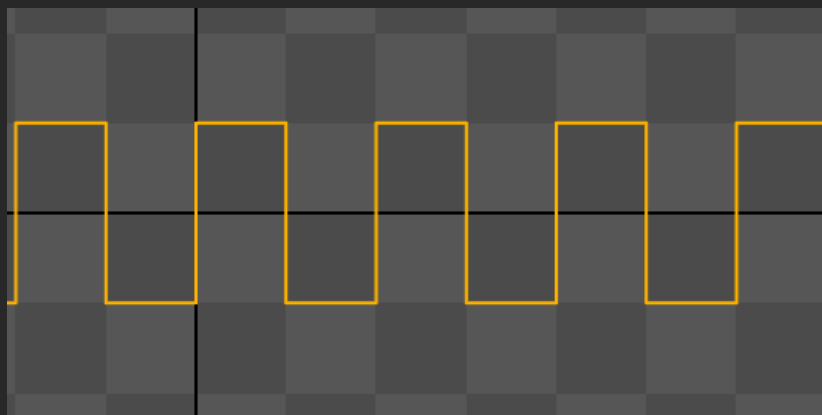
First we need to determine the area (in checkers pattern space) of the pixel under shading. Usually this is compute as an approximation to the true area. For that, most renderers make use of numeric differentiation -

they check the **uv** coordinates of the pattern in the the current pixel and subtract that from the **uv** coordinates at the neighbor pixels. That gives a good enough estimate of how much area of the pattern is compressed into that pixel. If you are using a simple pixel shader in a GPU, you can get that information with **dFdx()**, **ddx()** or **similar functions**. **If you are writing your own raytracer, you might want to employ ray differentials to keep track of this information.**

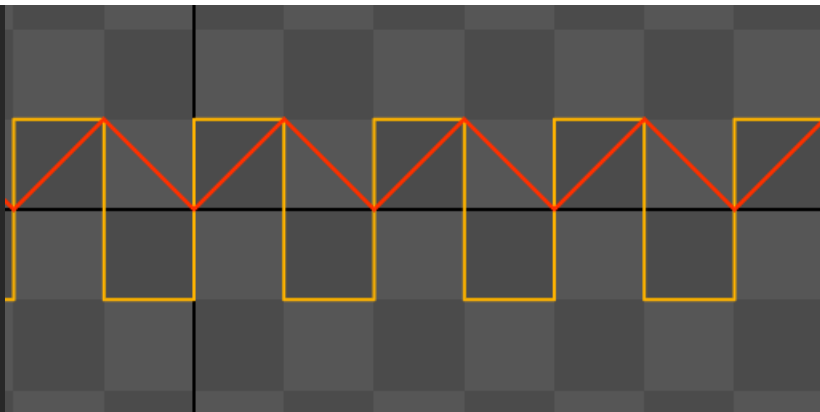
In either case, we'll assume such differentials are available and that they are good enough approximation to the shape of the kernel. One we have it, instead of **numerically integrate our pattern**, we'll try to do it analytically.

Integrating

As we mentioned earlier, the signal we want to integrate is a periodic square pulse, or a square wave, that oscillates between -1 and 1 along each unity of its domain.



Integration is accumulation of values, so as we start at zero and we start scanning the square signal we'll start accumulating values of 1, until we are done with the first square block. Then we'll start subtracting for a whole integer interval to reach zero again. The process will repeat forever, resulting in a signal with the same period as the square signal itself. Since we are integrating constant values, we know that the rate of increase and decreasing will be linear (a ramp). The peak will be the area of each square, which is 1. So with all this information, we can deduce that the integral of the square signal is the following triangular signal, displayed here in red:



We can codify such a signal easily, as shown bellow, although it can be sometimes convenient to have access both to the square and the triangular signals together:

```
// triangular signal
float tri( in float x )
{
    float h = fract(x*.5)-.5;
    return 1.-2.*abs(h);
}

// square (x) and triangular (y) signals
vec2 sqr_tri( in float x )
{
    float h = fract(x*.5)-.5;
    return vec2(-sign(h),1.-2.*abs(h));
}
```

With this at hand, we can now perform the actual integration. Remember that a definite integral of a function f with primitive F between points a and b is computed as $F(b)-F(a)$. So we are ready to put it all together:

1. Compute the kernel size based on our **uv** differentials, which we can call **ddx** and **ddy**: $w = \max\{|\text{ddx}|, |\text{ddy}|\}$
2. compute our points **a** as $\text{uv} - w/2$, **b** as $\text{uv} + w/2$
3. perform the integration of the **x** square signal and the **y** square signal, by using our triangular signal
4. divide it by the size of the kernel w so we get our normalize box filter with height $1/w$
5. evaluate our smooth xor function that we defined above to get our smooth checkers pattern

Note that integration (steps 3) is done for **x** and **y** independently, and the results is composed (step 5) through the xor pattern formula. In theory we should be integrating the xor pattern across a surface (it should be a double integral), but luckily for us the pattern integral separable:

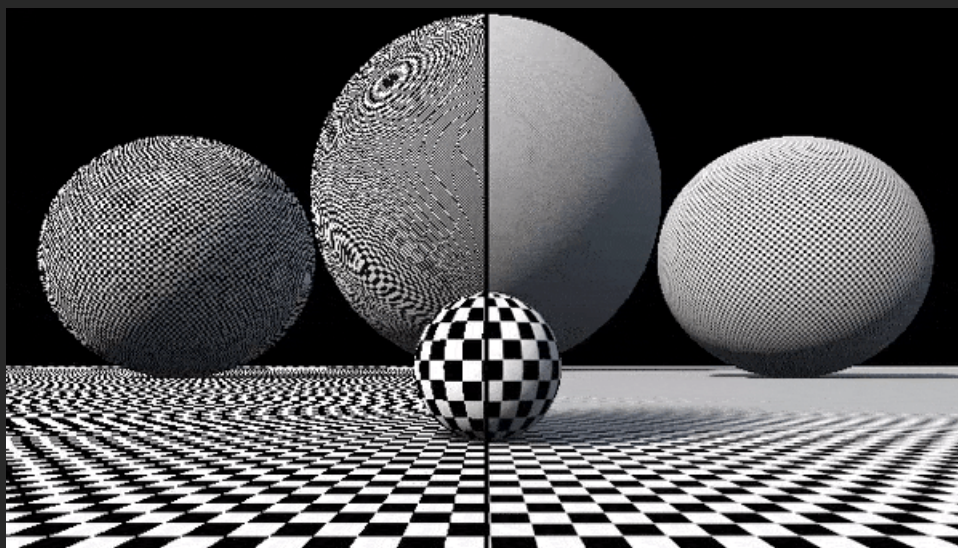
$$\int_{a_y}^{b_y} \int_{a_x}^{b_x} \left(\frac{1}{2} - \frac{1}{2} f(x)g(y) \right) \cdot dx dy = \frac{1}{2} - \frac{1}{2} \int_{a_x}^{b_x} f(x) dx \cdot \int_{a_y}^{b_y} g(y) dy$$

When we put all together, this is the resulting code:

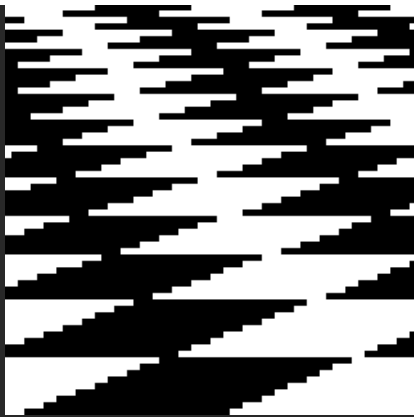
```
// square (x) and triangular (y) signals
float checkersGrad( in vec2 uv, in vec2 ddx, in vec2 ddy )
{
    vec2 w = max(abs(ddx), abs(ddy)) + 0.01;    // filter kernel
    vec2 i = (tri(uv+0.5*w)-tri(uv-0.5*w))/w;    // analytical integral (box filter)
    return 0.5 - 0.5*i.x*i.y;                    // xor pattern
}
```

As you can see, the signature of the function is the same as GLSL's textureGrad(), and it is intended to be used, indeed, in the same way.

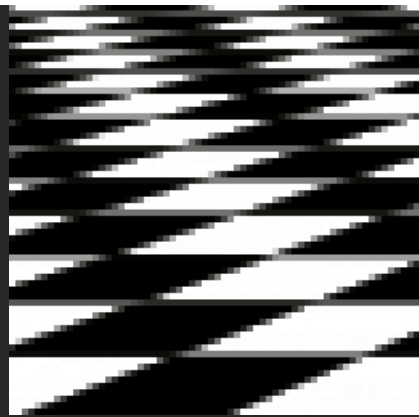
The image quality improvement speak by themselves, especially under camera animation:



Left is unfiltered, naive checkers pattern. Right is filtered as just described



Closeup, Unfiltered



Closeup, Filtered

You can find a reference implementation here: <https://www.shadertoy.com/view/XlcSz2>

[inigo quilez](https://iquilezles.org) - learning computer graphics since 1994